```
-- ============================================================
--  PROJECT  : Retail Sales Data Warehous
--  DATABASE : Microsoft SQL Server (gold schema)
--  TABLE    : gold.dim_customers
--  AUTHOR   : https://github.com/deepak015/-SQL-Sales-Analysis-Project
--  DATE     : May 2026
-- ============================================================
-- ============================================================
------ 1 Find total number of customers
-- ============================================================

SELECT
   COUNT(*) as Total_Customers
FROM gold.dim_customers

-- ============================================================
-- 2 Show all unique customer countries
-- ============================================================

SELECT
   DISTINCT country
from gold.dim_customers

-- ============================================================
-- 3 Find number of customers in each country
-- ============================================================

SELECT
```

```
    country,
    COUNT(*) AS Total_Customers
from gold.dim_customers
GROUP BY country

-- =============================================================
-- 4 Find male and female customer count
-- =============================================================

SELECT
    gender,
    COUNT(*) as Total_Customers
FROM gold.dim_customers
GROUP BY gender

-- =============================================================
-- 5 Find married and single customers count.
-- =============================================================

SELECT
    marital_status,
    COUNT(*) as Total_Customers
FROM gold.dim_customers
GROUP BY marital_status

-- =============================================================
-- 6 Find top 10 oldest customers.
-- =============================================================
```

```
SELECT
TOP 10
   customer_id,
   first_name,
   last_name,
   birthdate
FROM gold.dim_customers
WHERE birthdate IS NOT NULL
order by birthdate ASC

-- ============================================================
-- 7 Find customers born after 1968.
-- ============================================================

SELECT
   customer_id,
   first_name,
   last_name,
   birthdate
FROM gold.dim_customers
WHERE birthdate > '1968'

-- ============================================================
--8 Find average customer age
-- ============================================================

SELECT
```

```
   first_name,
   last_name,
   AVG(DATEDIFF(year,birthdate, GETDATE())) as Average_Age
FROM gold.dim_customers
WHERE birthdate IS NOT NULL
GROUP BY first_name,
last_name

-- ============================================================
-- 9 Find customers whose first name starts with 'A'
-- ============================================================

SELECT
   first_name
FROM gold.dim_customers
WHERE first_name LIKE 'A%'

-- ============================================================
--10 Find country having maximum customers
-- ============================================================

SELECT
TOP 1
   country,
   COUNT(*) as total_customers
FROM gold.dim_customers
GROUP by country
ORDER BY COUNT(*) desc
```

```
--  TABLE    : gold.dim_product

-- 1 Find total number of products.

SELECT
    COUNT(*) AS TotalProducts
FROM gold.dim_products;

   --2 Find all product categories

   SELECT
      COUNT(DISTINCT category) AS TotalCategories
   FROM gold.dim_products;

   -- 3 Find number of products in each category

   SELECT
   category,
      COUNT(*) AS ProductsInCategory
   FROM gold.dim_products
   GROUP BY category;

   --4 Find most expensive product.

   SELECT TOP 1
      product_name,
      cost
   FROM gold.dim_products
```

```
ORDER BY cost DESC;

-- 5 Find least expensive product

SELECT
 TOP 1
   product_name,
   cost
FROM gold.dim_products
ORDER BY cost ASC;

-- 6 Find products that require maintenance

SELECT
   product_name,
   maintenance
FROM gold.dim_products
WHERE maintenance = 'Yes';

--7 Find products launched after 2010

SELECT
   product_name,
   start_date
FROM gold.dim_products
where start_date > '2010-12-31';

-- 8 Find total products in each product line
```

```
SELECT
product_line,
    COUNT(*) AS TotalProductsInLine
FROM gold.dim_products
GROUP BY product_line;

--9 Find average product cost.
SELECT
    AVG(cost) AS AverageProductCost
FROM gold.dim_products;

-- 10 Find category with highest products

SELECT TOP 1
category,
    COUNT(*) AS ProductsInCategory
FROM gold.dim_products
GROUP BY category
ORDER BY ProductsInCategory DESC;

--  TABLE    : gold.dim_orders

-- 1 Find total sales amount.

SELECT
    SUM(sales_amount) AS total_sales_amount
FROM
```

```
  gold.fact_sales;

-- 2 Find total quantity sold

SELECT
  SUM(quantity) AS total_quantity_sold
FROM
  gold.fact_sales;

-- 3 Find total number of orders.

SELECT
  COUNT(DISTINCT order_number) AS total_orders
FROM
  gold.fact_sales;

-- 4 Find average sales amount

SELECT
  AVG(sales_amount) AS average_sales_amount
FROM
  gold.fact_sales;

-- 5 Find highest sales amount.

SELECT
  MAX(sales_amount) AS highest_sales_amount
FROM
```

```
   gold.fact_sales;

--6 Find lowest sales amount.

SELECT
   MIN(sales_amount) AS lowest_sales_amount
FROM
   gold.fact_sales;

-- 7 Find monthly sales trend.

SELECT
   YEAR(order_date) as sales_year,
   MONTH(order_date) as sales_month,
   SUM(sales_amount) AS monthly_sales
FROM
   gold.fact_sales
GROUP BY
   YEAR(order_date),
   MONTH(order_date)
ORDER BY
sales_year,
   sales_month;

-- 8 Find yearly sales trend.

SELECT
   YEAR(order_date) as sales_year,
```

```
    SUM(sales_amount) AS yearly_sales
FROM
    gold.fact_sales
GROUP BY
    YEAR(order_date)
ORDER BY
    sales_year;

-- 9 Find sales for each month.

SELECT
    MONTH(order_date) as sales_month,
    SUM(sales_amount) AS monthly_sales
FROM
    gold.fact_sales
GROUP BY
    MONTH(order_date)
ORDER BY
    sales_month;

-- 10 Find busiest sales month.

SELECT
TOP 1
    MONTH(order_date) as sales_month,
    SUM(sales_amount) AS monthly_sales
FROM
    gold.fact_sales
```

```
GROUP BY
    MONTH(order_date)
ORDER BY
    monthly_sales DESC;
```

TABLE: customers+orders

```
-- 1 Find top 10 customers by total spending.

SELECT
    TOP 10
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(o.sales_amount) AS total_spending
FROM
    gold.dim_customers c
LEFT JOIN
    gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name
ORDER BY
    total_spending DESC;

-- 2 Find customers who placed maximum orders
```

```
SELECT top 1
   c.customer_id,
   c.first_name,
   COUNT(o.order_number) AS max_orders
FROM
   gold.dim_customers c
LEFT JOIN
   gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY
   c.customer_id,
   c.first_name
ORDER BY
   max_orders DESC


-- 3 Find average revenue per customer.
SELECT
   c.customer_id,
   c.first_name,
   AVG(o.sales_amount) AS avg_revenue
FROM
   gold.dim_customers c
LEFT JOIN
   gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY
   c.customer_id,
   c.first_name
ORDER BY
   avg_revenue DESC;
```

```sql
-- 4 Find country-wise sales revenue.
SELECT
   c.country,
   SUM(o.sales_amount) AS country_sales_revenue
FROM
   gold.dim_customers c
LEFT JOIN
   gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY
   c.country
ORDER BY
   country_sales_revenue DESC;

-- 5 Find gender-wise sales analysis
SELECT
   c.gender,
   SUM(o.sales_amount) AS gender_wise_sales
FROM
   gold.dim_customers c
LEFT JOIN
   gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY
   c.gender
ORDER by
   gender_wise_sales desc;

-- 6 Find married vs single customer sales
```

```
SELECT
   c.marital_status,
   SUM(o.sales_amount) AS marital_status_sales
FROM
   gold.dim_customers c
LEFT JOIN
   gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY
   c.marital_status
ORDER BY
   marital_status_sales DESC;

-- 7 Find customers who purchased most quantity

SELECT top 1
   c.customer_id,
   c.first_name,
   SUM(o.quantity) AS total_quantity
FROM
   gold.dim_customers c
LEFT JOIN
   gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY
   c.customer_id,
   c.first_name
ORDER BY
   total_quantity DESC;
```

```
-- 8 Find top spending female customers.

  SELECT
    TOP 1
    c.customer_key,
    c.first_name,
    c.gender,
    SUM(o.sales_amount) AS total_spending
  FROM
    gold.dim_customers c
  LEFT JOIN
    gold.fact_sales o ON c.customer_id = o.customer_key
  WHERE c.gender ='female'
  GROUP BY c.customer_key,c.gender,c.first_name
  ORDER BY total_spending DESC;

-- 9 Find customers with more than 5 orders

  SELECT
    TOP 5
    c.customer_id,
    c.first_name,
    COUNT(o.order_number) AS total_orders
  FROM
    gold.dim_customers c
  LEFT JOIN
    gold.fact_sales o ON c.customer_id = o.customer_key
```

```
    GROUP BY
      c.customer_id,
      c.first_name
    ORDER BY
      total_orders DESC

    -- 10 Find customer contribution percentage in total sales.

    SELECT
      c.customer_id,
      c.first_name,
      SUM(o.sales_amount) AS customer_sales,
      ROUND
        (
      (SUM(o.sales_amount) *100.0 / SUM(SUM(o.sales_amount)) OVER ()) ,2) AS
contribution_percentage
      FROM
        gold.dim_customers c
      LEFT JOIN
        gold.fact_sales o ON c.customer_id = o.customer_key
      GROUP BY
        c.customer_id,
        c.first_name
      ORDER BY
        contribution_percentage DESC;
```

TABLE: Product + Orders

```
-- 1 Find top 10 best-selling products.

  SELECT
    TOP 10
    p.product_id,
    p.product_name,
    SUM(o.quantity) AS best_selling_product
  FROM gold.dim_products p
  LEFT JOIN
    gold.fact_sales  o on p.product_key=o.product_key
  GROUP BY p.product_id,
      p.product_name
    ORDER BY best_selling_product desc

  -- 2 Find least-selling products.

    SELECT
    TOP 10
    p.product_id,
    p.product_name,
    SUM(o.quantity) AS least_selling_product
  FROM gold.dim_products p
  LEFT JOIN
    gold.fact_sales  o on p.product_key=o.product_key
  GROUP BY p.product_id,
      p.product_name
    ORDER BY least_selling_product asc
```

```
-- 3 Find category-wise revenue

SELECT
   p.category,
   SUM(o.sales_amount) category_wise_revenue
FROM gold.dim_products p
LEFT JOIN
   gold.fact_sales  o on p.product_key=o.product_key
GROUP BY p.category
   ORDER BY category_wise_revenue desc

-- 4 Find subcategory-wise revenue

SELECT
   p.subcategory,
   SUM(o.sales_amount) category_wise_revenue
FROM gold.dim_products p
LEFT JOIN
   gold.fact_sales  o on p.product_key=o.product_key
GROUP BY p.subcategory
   ORDER BY category_wise_revenue desc

-- 5 Find total quantity sold for each product

SELECT
   p.product_id,
   p.product_name,
   SUM(o.quantity) total_quantity_sold
```

```
FROM gold.dim_products p
LEFT JOIN
  gold.fact_sales  o on p.product_key=o.product_key
GROUP BY p.product_id,
     p.product_name
  ORDER BY total_quantity_sold desc

-- 6 Find highest revenue generating category.

SELECT
  p.category,
  SUM(o.sales_amount) category_wise_revenue
FROM gold.dim_products p
LEFT JOIN
  gold.fact_sales  o on p.product_key=o.product_key
GROUP BY p.category
  ORDER BY category_wise_revenue desc

-- 7 Find top 5 products by sales amount.

SELECT
  TOP 5
  p.product_name,
  SUM(o.sales_amount) total_sales_amount
FROM gold.dim_products p
LEFT JOIN
  gold.fact_sales  o on p.product_key=o.product_key
GROUP BY p.product_name
```

```
ORDER BY total_sales_amount desc

-- 8 Find products never sold

SELECT
   p.product_name
FROM gold.dim_products p
LEFT JOIN
   gold.fact_sales  o on p.product_key=o.product_key
WHERE o.product_key IS NULL

-- 9 Find average sales per product

SELECT
   p.product_name,
   AVG(o.sales_amount) average_sales_per_product
FROM gold.dim_products p
LEFT JOIN
   gold.fact_sales  o on p.product_key=o.product_key
GROUP BY p.product_name
ORDER BY average_sales_per_product desc

-- 10 Find percentage contribution of each category in total sales.

SELECT
   p.category,
   SUM(o.sales_amount) category_sales,
```

```
    SUM(o.sales_amount) * 100.0 / (SELECT SUM(sales_amount) FROM gold.fact_sales)
AS percentage_contribution
    FROM
      gold.dim_products p
  LEFT JOIN
    gold.fact_sales  o on p.product_key=o.product_key
  GROUP BY p.category
  ORDER BY percentage_contribution desc
```

Table; Date and Sales

```
-- 1 Find total sales year-wise.

  SELECT
    YEAR(order_date) AS SalesYear,
    SUM(sales_amount) AS TotalSales
  FROM
    gold.fact_sales
  GROUP BY
    YEAR(order_date)
  ORDER BY
    SalesYear;

  -- 2 Find total sales quarter-wise.

  SELECT
    YEAR(order_date) AS SalesYear,
```

```
    DATEPART(QUARTER, order_date) AS SalesQuarter,
    SUM(sales_amount) AS TotalSales
FROM
    gold.fact_sales
GROUP BY
    YEAR(order_date),
    DATEPART(QUARTER, order_date)
ORDER BY
    SalesYear,
    SalesQuarter;

    -- 3 Find total sales month-wise.

    SELECT
        YEAR(order_date) AS SalesYear,
        MONTH(order_date) AS SalesMonth,
        SUM(sales_amount) AS TotalSales
    FROM
        gold.fact_sales
    GROUP BY
        YEAR(order_date),
        MONTH(order_date)
    ORDER BY
        SalesYear,
        SalesMonth;

    --4 Find orders placed on weekends
```

```
SELECT
    YEAR(order_date) AS SalesYear,
    DATEPART(WEEKDAY, order_date) AS Weekday,
    SUM(sales_amount) AS TotalSales
FROM
    gold.fact_sales
WHERE
    DATEPART(WEEKDAY, order_date) IN (1, 7) -- Assuming 1 = Sunday and 7 = Saturday
GROUP BY
    YEAR(order_date),
    DATEPART(WEEKDAY, order_date)
ORDER BY
    SalesYear,
    Weekday;

-- 5 Find orders placed in December.

SELECT
    YEAR(order_date) AS SalesYear,
    MONTH(order_date) AS SalesMonth,
    SUM(sales_amount) AS TotalSales
FROM
    gold.fact_sales
WHERE
    MONTH(order_date) = 12
GROUP BY
    YEAR(order_date),
```

```
    MONTH(order_date)
  ORDER BY
    SalesYear,
    SalesMonth;

  -- 6 Find first order date.

  SELECT
    MIN(order_date) AS FirstOrderDate
  FROM
    gold.fact_sales;

-- 7 Find latest order date.

SELECT
    MAX(order_date) AS LatestOrderDate
  FROM
    gold.fact_sales;

-- 8 Find average orders per month

SELECT
  YEAR(order_date) AS SalesYear,
  MONTH(order_date) AS SalesMonth,
  AVG(sales_amount) AS AverageSales
FROM
  gold.fact_sales
GROUP BY
```

```
    YEAR(order_date),
    MONTH(order_date)
  ORDER BY
    SalesYear,
    SalesMonth;

  -- 9 Find sales growth year by year

  WITH YearlySales AS (
  SELECT
    YEAR(order_date) AS SalesYear,
    SUM(sales_amount) AS TotalSales
  FROM
    gold.fact_sales
  GROUP BY
    YEAR(order_date)
  )

  SELECT
    SalesYear,
    TotalSales,
    LAG(TotalSales) OVER (ORDER BY SalesYear) AS PreviousYearSales,
    TotalSales- LAG(TotalSales) OVER (ORDER BY SalesYear) AS SalesGrowth,
    ROUND(
      ((LAG(TotalSales) OVER (ORDER BY SalesYear)) * 100.)/ Lag(TotalSales) OVER
(ORDER BY SalesYear),2 ) AS SalesGrowthPercentage
    FROM
      YearlySales
```

```
-- 10 Find running total of sales.

  SELECT
  order_date,
    sales_amount,
    SUM(sales_amount) OVER (ORDER BY order_date) AS RunningTotal
  FROM
    gold.fact_sales
```

Table: Window Function

```
-- 1 Rank customers based on total spending.

  SELECT
    customer_id,
    first_name,
    SUM(sales_amount) AS total_spending,
    RANK() OVER (ORDER BY SUM(sales_amount) DESC) AS spending_rank
  FROM
    gold.fact_sales fs
  LEFT JOIN
    gold.dim_customers dc ON fs.customer_key = dc.customer_key
  GROUP BY
    customer_id,
    first_name

  -- 2 Rank products based on revenue.
```

```
SELECT
   product_id,
   product_name,
   SUM(sales_amount) AS total_revenue,
   RANK() OVER (ORDER BY SUM(sales_amount) DESC) AS revenue_rank
FROM
   gold.fact_sales fs
LEFT JOIN
   gold.dim_products dc ON fs.customer_key = dc.product_key
GROUP BY
   product_id,
   product_name

-- 3 Find previous month sales using


WITH PreviousMonthSales AS (
   SELECT
      YEAR(order_date) AS order_year,
      MONTH(order_date) AS order_month,
      SUM(sales_amount) AS total_sales
   FROM
      gold.fact_sales
   GROUP BY
      YEAR(order_date),
      MONTH(order_date)
      )
```

```
    SELECT
        order_year
        order_month,
        total_sales,
        LAG(total_sales) OVER (ORDER BY order_year, order_month) AS
previous_month_sales
    FROM
        PreviousMonthSales

-- 4 Find next month sales using LEAD().

WITH NextMonthSales AS (
SELECT
    YEAR(order_date) AS order_year,
    MONTH(order_date) AS order_month,
    SUM(sales_amount) AS total_sales
FROM
    gold.fact_sales
GROUP BY
    YEAR(order_date),
    MONTH(order_date)
    )

SELECT
    order_year,
    order_month,
    total_sales,
```

```
        LEAD(total_sales) OVER (ORDER BY order_year, order_month) AS
next_month_sales
    FROM
        NextMonthSales


-- 5 Find top 3 products in each category.


WITH RankedProducts AS (
    SELECT
    product_id,
    category,
    product_name,
    SUM(sales_amount) AS total_revenue
FROM
    gold.fact_sales fs
LEFT JOIN
    gold.dim_products dp ON fs.product_key = dp.product_key
GROUP BY
    product_id,
    category,
    product_name
    )
,products_with_rank AS (
SELECT
    product_id,
    category,
    product_name,
    total_revenue,
```

```
    RANK() OVER (PARTITION BY category ORDER BY total_revenue DESC) AS
category_rank
  FROM
    RankedProducts)

    SELECT *
    FROM products_with_rank
    WHERE category_rank <= 3

  -- 6 Find dense rank of countries by sales.

    WITH CountrySales AS (
    SELECT
      country,
      SUM(sales_amount) AS total_sales
    FROM
      gold.fact_sales fs
    LEFT JOIN
      gold.dim_customers dc ON fs.customer_key = dc.customer_key
    GROUP BY
      country
    )

    SELECT
      country,
      total_sales,
      DENSE_RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
    FROM
```

```
    CountrySales

-- 7 Find cumulative sales using SUM() OVER().

  SELECT
    order_date,
    SUM(sales_amount) AS daily_sales,
    SUM(SUM(sales_amount)) OVER (ORDER BY order_date) AS cumulative_sales
  FROM
    gold.fact_sales
  GROUP BY
    order_date

-- 8 Find moving average of sales.

  WITH DailySales AS (
  SELECT
    YEAR(order_date) AS order_year,
    MONTH(order_date) AS order_month,
    SUM(sales_amount) AS total_sales
  FROM
    gold.fact_sales
  GROUP BY
    YEAR(order_date),
    MONTH(order_date)
    )

  SELECT
```

```
    order_year,
    order_month,
    total_sales,
    AVG(total_sales) OVER (ORDER BY order_year, order_month ROWS BETWEEN 2
PRECEDING AND CURRENT ROW) AS moving_average
FROM
    DailySales

-- 8 Find customer order sequence number

SELECT
    customer_id,
    order_number,
    RANK() OVER (PARTITION BY customer_id ORDER BY order_date) AS
order_sequence
FROM
    gold.fact_sales fs
    LEFT JOIN
        gold.dim_customers dc ON fs.customer_key = dc.customer_key
GROUP BY
    customer_id,
    order_number,
    order_date

-- 10 Find highest sales in each year

WITH YearlyProductSales AS (
SELECT
    YEAR(order_date) AS order_year,
```

```
        product_key,
        SUM(sales_amount) AS highest_sales
        FROM
          gold.fact_sales
        GROUP BY
          YEAR(order_date),
          product_key
          )
      , RankedYearlySales AS (
        SELECT
          order_year,
          product_key,
          highest_sales,
          RANK() OVER (PARTITION BY order_year ORDER BY highest_sales DESC) AS
sales_rank
        FROM
          YearlyProductSales
          )

        SELECT *
          FROM RankedYearlySales
          WHERE sales_rank = 1
```